

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

ассистент

должность, уч. степень, звание

подпись, дата

И.Д. Свеженин

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №2

Многопоточное программирование

по курсу: Кроссплатформенное программирование

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков

инициалы, фамилия

Санкт-Петербург 2025

СОДЕРЖАНИЕ

1. Задание	2
2. Ход работы.....	3
3. Результат работы программы.....	4
4. Вывод.....	6
5. Листинг с кодом программы.....	7

1. Задание

Для выполнения первого варианта второй лабораторной работы программа должна реализовывать:

1. Функцию сложения чисел, то есть `int addUp(Stream numbers);`;
2. Функцию, которая получает исполнителя и возвращает список строк, содержащих имена и место происхождения.
3. Функцию, которая получает альбомы и возвращает список альбомов, содержащих не более трех произведений.

2. Ход работы

В ходе выполнения лабораторной работы была разработана программа на языке Kotlin, которая демонстрирует основы многопоточного и потокового программирования. В программе реализована функция сложения чисел, принимающая поток целых значений и вычисляющая их сумму. Далее была создана функция для обработки потока исполнителей, которая формирует список строк с их именами и местом происхождения. Кроме того, была реализована функция для работы с потоком альбомов, которая отбирает только те, что содержат не более трёх треков. Для организации данных использовались классы Artist, Album и Track, что позволило структурировать информацию и упростить обработку коллекций.

Работа программы основана на применении потоков Stream, позволяющих выполнять фильтрацию, преобразование и агрегацию данных в функциональном стиле. Использование лямбда-выражений сделало реализацию компактной и наглядной, а комбинация операций map, filter и sum продемонстрировала возможности параллельной и последовательной обработки данных. Таким образом, в ходе лабораторной работы были закреплены навыки работы с потоками в Kotlin, а также рассмотрены приёмы функциональной обработки коллекций в контексте многопоточного программирования.

3. Результат работы программы

В качестве результата работы программы представлен вывод значений в терминал. На рисунках 1-2 показан пример работы программы с различными значениями.

```
println("Сумма чисел: ${addUp( numbers = listOf(1,2,3,4,5).stream())}")

val artists = Stream.of(
    ...values = Artist( name = "The Beatles", origin = "Liverpool"),
    Artist( name = "Queen", origin = "London"),
    Artist( name = "Nirvana", origin = "Seattle"),
    Artist( name = "Madonna", origin = "Michigan"),
    Artist( name = "Elvis Presley", origin = "Mississippi"),
    Artist( name = "Bob Marley", origin = "Jamaica")
)

val artistsInfo = getArtistsInfo(artists)
println("\nИнформация об артистах:")
artistsInfo.forEach { println(it) }

val albums = Stream.of(
    ...values = Album( title = "Album 1", artist = "Artist A", tracks = listOf(
        Track( title = "Song 1", duration = 180),
        Track( title = "Song 2", duration = 240)
    )),
    Album( title = "Album 2", artist = "Artist B", tracks = listOf(
        Track( title = "Track 1", duration = 200),
        Track( title = "Track 2", duration = 220),
        Track( title = "Track 3", duration = 190)
    )),
    Album( title = "Album 3", artist = "Artist C", tracks = listOf(
        Track( title = "Intro", duration = 60),
        Track( title = "Verse", duration = 180),
        Track( title = "Chorus", duration = 200),
        Track( title = "Outro", duration = 90)
    )),
    Album( title = "Single", artist = "Artist D", tracks = listOf(
        Track( title = "Main Track", duration = 300)
    )),
    Album( title = "Greatest Hits", artist = "Artist E", tracks = listOf(
        Track( title = "Hit 1", duration = 240),
        Track( title = "Hit 2", duration = 210),
        Track( title = "Hit 3", duration = 195),
        Track( title = "Hit 4", duration = 220),
        Track( title = "Hit 5", duration = 230)
    ))
)
```

Рисунок 1 – Вводимые значения

```
Сумма чисел: 15

Информация об артистах:
The Beatles - Liverpool
Queen - London
Nirvana - Seattle
Madonna - Michigan
Elvis Presley - Mississippi
Bob Marley - Jamaica
|
Альбомы с не более чем 3 треками:
Album 1 - Artist A (2 треков)
Album 2 - Artist B (3 треков)
Single - Artist D (1 треков)
```

Рисунок 2 – Результат работы программы

4. Вывод

В результате выполнения лабораторной работы была создана программа на языке Kotlin, которая демонстрирует применение потокового подхода к обработке данных. Реализованные функции позволили выполнять суммирование чисел, формирование информации об исполнителях и фильтрацию альбомов по количеству треков.

Работа показала, что использование потоков и функциональных операций значительно упрощает обработку коллекций и делает код более читаемым и удобным для сопровождения. Таким образом, поставленные задачи были успешно решены, а лабораторная работа помогла закрепить навыки многопоточного и функционального программирования.

5. Листинг с кодом программы

В данном разделе представлен исходный код моей части программы:

```
//Kotlin

import java.util.stream.Stream
import java.util.stream.Stream.*

fun main() {

    println("Сумма чисел: ${addUp(listOf(1,2,3,4,5).stream())}")

    val artists = Stream.of(
        Artist("The Beatles", "Liverpool"),
        Artist("Queen", "London"),
        Artist("Nirvana", "Seattle"),
        Artist("Madonna", "Michigan"),
        Artist("Elvis Presley", "Mississippi"),
        Artist("Bob Marley", "Jamaica")
    )

    val artistsInfo = getArtistsInfo(artists)
    println("\nИнформация об артистах:")
    artistsInfo.forEach { println(it) }

    val albums = Stream.of(
        Album("Album 1", "Artist A", listOf(
            Track("Song 1", 180),
            Track("Song 2", 240)
        )),
        Album("Album 2", "Artist B", listOf(
```



```

        Track("Track 1", 200),
        Track("Track 2", 220),
        Track("Track 3", 190)
    )),
    Album("Album 3", "Artist C", listOf(
        Track("Intro", 60),
        Track("Verse", 180),
        Track("Chorus", 200),
        Track("Outro", 90)
    )),
    Album("Single", "Artist D", listOf(
        Track("Main Track", 300)
    )),
    Album("Greatest Hits", "Artist E", listOf(
        Track("Hit 1", 240),
        Track("Hit 2", 210),
        Track("Hit 3", 195),
        Track("Hit 4", 220),
        Track("Hit 5", 230)
    ))
)

```

```

val shortAlbums = getAlbumsWithMaxThreeTracks(albums)
println("\nАльбомы с не более чем 3 треками:")
shortAlbums.forEach { album ->
    println("${album.title} - ${album.artist} (${album.tracks.size} треков)")
}
}

```

```

fun addUp(numbers: Stream<Int>): Int {

```

```
    return numbers.mapToInt { it }.sum()
}
```

```
data class Artist(
    val name: String,
    val origin: String
)
```

```
fun getArtistsInfo(artists: Stream<Artist>): List<String> {
    return artists
        .map { artist -> "${artist.name} - ${artist.origin}" }
        .toList()
}
```

```
data class Track(
    val title: String,
    val duration: Int
)
```

```
data class Album(
    val title: String,
    val artist: String,
    val tracks: List<Track>
)
```

```
fun getAlbumsWithMaxThreeTracks(albums: Stream<Album>): List<Album> {
    return albums
        .filter { album -> album.tracks.size <= 3 }
        .toList()
}
```